ok, tuttavia con questa soluzione non mi e' chiaro dove sono dichiarati i mount standard del linguaggio e come vengono gestiti nel piano dei volumi e dei mount proposto

Hai ragione: nella proposta precedente mancava un elemento esplicito. Ho indicato “configurazione del linguaggio”, senza definire il file che contiene i mount standard né il suo rapporto con gli script di setup.

La soluzione che propongo è dichiarare i mount standard in `langs/Python/runtime.yaml`, usando lo stesso schema del file runtime del progetto, ma con responsabilità e regole di composizione diverse.

1. Due livelli di dichiarazione

```
docker-dev/
└── langs/
    └── Python/
        ├── runtime.yaml         # default e mount standard Python
        ├── templates/
        └── bin/
            ├── 10.setup-volumes.sh
            ├── 20.build.sh
            ├── 30.services.sh
            └── 40.run.sh

<PROJECT_DIR>/
└── .docker-dev/
    └── runtime.yaml             # aggiunte specifiche del progetto
```

File	Contenuto
`langs/Python/runtime.yaml`	Esigenze standard dell’ambiente Python
`<PROJECT_DIR>/.docker-dev/runtime.yaml`	Esigenze aggiuntive del progetto
Piano risolto	Composizione validata dei due livelli, con nomi e percorsi definitivi

Il file del progetto non deve ripetere i mount standard.

2. Dichiarazione standard Python

Questo è uno schema proposto, da implementare nel resolver; i segnaposto non sono espressioni Compose o Template::Toolkit.

```
# docker-dev/langs/Python/runtime.yaml
schema_version: 1

volumes:
  python-venv:
    name: "${context.volume_name}"
    lifecycle: external
    preparation:
      strategy: language-hook

  uv-python:
    name: uv-python
    lifecycle: external
    preparation:
      strategy: language-hook

mounts:
  - id: project-source
    type: bind
    source: "${context.project_dir}"
    target: /app
    read_only: false
    protected: true

  - id: python-venv
    type: volume
    source: python-venv
    target: /app/.venv
    read_only: false
    protected: true

  - id: uv-python
    type: volume
    source: uv-python
    target: "/home/${context.host_user}/.local/share/uv/python"
    read_only: false
    protected: true
```

Qui distinguiamo due concetti:

- `volumes` dichiara le risorse persistenti: nome effettivo e responsabilità di preparazione;
- `mounts` dichiara dove quelle risorse, o directory host, vengono collegate nel container.

Per un mount di tipo `volume`, `source` è una chiave logica della sezione `volumes`, non il nome Docker definitivo.

`protected: true` significa che il progetto non può sostituire quel mount implicitamente.

Contesto utilizzato

Il main fornisce al resolver valori già determinati:

```
project_dir: /home/utente/REP/DEV/Python/foo
host_user: utente
volume_name: venv.dev.python.foo
```

In particolare, `volume_name` deve conservare l'eventuale override `VOLUME_NAME`, anziché ricostruire sempre il nome dalla convenzione.

Il resolver espande soltanto i segnaposto previsti dallo schema, senza valutare codice o espressioni shell.

3. Aggiunte del progetto

Il progetto può dichiarare:

```
# <PROJECT_DIR>/.docker-dev/runtime.yaml
schema_version: 1

mounts:
  - id: datasets
    type: bind
    source: ./datasets
    target: /datasets
    read_only: true
```

Il resolver compone:

```
3 mount standard Python
        +
1 mount aggiuntivo del progetto
        ↓
4 mount nel runtime risolto
```

Il percorso `./datasets` viene risolto rispetto alla directory del progetto.

Se il progetto tenta di dichiarare un altro mount con destinazione `/app/.venv`, la composizione fallisce: non sostituisce silenziosamente il mount standard.

4. Come vengono inseriti nel piano

Il piano risolto dovrebbe mantenere separate le risorse da preparare e i mount da applicare, conservando anche la provenienza delle dichiarazioni.

Un esempio abbreviato:

```
schema_version: 1
```

```
volume_resources:
  - id: python-venv
    docker_name: venv.dev.python.foo
    lifecycle: external
    preparation:
      owner: language
      hook: langs/Python/bin/10.setup-volumes.sh
    origin: language

  - id: uv-python
    docker_name: uv-python
    lifecycle: external
    preparation:
      owner: language
      hook: langs/Python/bin/10.setup-volumes.sh
    origin: language

container_mounts:
  - id: project-source
    type: bind
    source: /home/utente/REP/DEV/Python/foo
    target: /app
    read_only: false
    origin: language

  - id: python-venv
    type: volume
    resource_id: python-venv
    source: venv.dev.python.foo
    target: /app/.venv
    read_only: false
    origin: language

  - id: uv-python
    type: volume
    resource_id: uv-python
    source: uv-python
    target: /home/utente/.local/share/uv/python
    read_only: false
    origin: language

  - id: datasets
    type: bind
    source: /home/utente/REP/DEV/Python/foo/datasets
    target: /datasets
    read_only: true
    origin: project
```

Questo piano è un formato interno di `docker-dev`, non un documento da passare direttamente a Compose.

La separazione consente di rispondere a due domande diverse:

Domanda	Sezione del piano
Quali volumi devono esistere ed essere inizializzati?	`volume_resources`
Quali mount deve ricevere il container?	`container_mounts`

5. Ruolo di `10.setup-volumes.sh`

Nella prima implementazione manterrei la preparazione specifica Python dentro `10.setup-volumes.sh`, ma lo adatterei affinché consumi il piano.

Il contratto diventa, per esempio:

```
10.setup-volumes.sh --plan "$PLAN_FILE"
```

Lo script:

1. legge le risorse assegnate alla preparazione del linguaggio;
2. usa i nomi effettivi presenti nel piano;
3. applica la logica Python di creazione e inizializzazione;
4. non decide quali mount applicare al container.

La conoscenza specifica — ad esempio ownership richiesta dal virtualenv — resta nello script Python. La dichiarazione dell'esistenza del volume e il suo nome, invece, hanno una sola fonte nel modello risolto.

Per evitare di eseguire due volte lo stesso hook, il runner lo invoca una volta per fase, passandogli tutte le risorse di sua competenza.

Volumi aggiuntivi del progetto

Non devono essere assegnati automaticamente allo script Python. Per questi servirà una strategia separata, per esempio:

- creazione generica senza inizializzazione;
- inizializzazione dichiarativa supportata dal tool;
- hook esplicito del progetto.

Questa scelta resta da definire. È distinta dalla gestione dei due volumi standard, per i quali abbiamo già uno script di preparazione.

6. Ruolo di `40.run.sh`

`40.run.sh` non deve più contenere i tre mount Python hardcoded.

Consuma invece `container_mounts` tramite il backend scelto:

```
container_mounts
      ├─ legacy  → array di argomenti mount
```

```
        └─ Compose → services.dev.volumes
```

Il backend Compose usa inoltre `volume_resources` per generare le dichiarazioni top-level dei volumi. Il backend legacy usa i nomi effettivi direttamente negli argomenti.

Di conseguenza, lo script conserva la responsabilità di esecuzione, ma non una seconda copia della configurazione dei mount.

7. Sequenza completa

```
docker-dev.sh
    │
    ├─ determina progetto, linguaggio e contesto
    │
    ├─ resolver runtime Perl
    │    ├─ legge langs/Python/runtime.yaml
    │    ├─ legge .docker-dev/runtime.yaml, se presente
    │    ├─ risolve nomi e percorsi
    │    ├─ verifica riferimenti e conflitti
    │    └─ produce il piano
    │
    ├─ 10.setup-volumes.sh
    │    └─ prepara le risorse Python indicate nel piano
    │
    ├─ 20.build.sh
    │
    ├─ 30.services.sh
    │    └─ risolve e prepara i servizi opzionali
    │
    └─ 40.run.sh
         └─ applica i mount risolti tramite legacy o Compose
```

Quindi il punto da aggiungere alla CR è preciso: `langs/<tipo>/runtime.yaml` diventa la fonte dichiarativa dei mount standard; il resolver li compone con le aggiunte del progetto; gli script di setup preparano le risorse indicate dal piano e lo script di run applica i mount, senza ridefinirli.